

Einführung: Der Software-Lebenszyklus (SDLC)

Von der Idee zur gewarteten Anwendung — und wie Spec-Driven Development diesen Weg geht.

Lernziele

Nach dieser Einheit können Sie ...

- die sechs Phasen des Software-Lebenszyklus benennen und in eigenen Worten erklären,
- begründen, warum **WAS** (Anforderung) und **WIE** (Entwurf) getrennte Phasen sind,
- zwei Ebenen der Prüfung unterscheiden (funktioniert der Baustein? — erfüllt das Ganze die Anforderung?),
- den SDLC als **Kreislauf** statt als Linie beschreiben,
- **Ihr eigenes** Projekt den Phasen zuordnen.

1 | Worum es geht

Kein ernsthaftes Softwareprojekt springt von der Idee direkt zum fertigen Programm. Zwischen „Ich hätte gern ...“ und „läuft im Betrieb“ liegt eine feste Abfolge von Phasen — der **Software Development Life Cycle (SDLC)**. Er ist kein Werkzeug und keine Methode, sondern die *Landkarte* der Tätigkeiten, die jedes Projekt durchläuft — ob man sie benennt oder nicht.

Wahrscheinlich sind Sie diesen Weg längst gegangen, nur ohne die Vokabeln. Das Ziel dieser Einheit ist, den Phasen ihre Namen zu geben — damit Sie bei jedem künftigen Projekt wissen, *wo* Sie gerade stehen und *was* als Nächstes ansteht.

2 | Die sechs Phasen

#	Phase	Leitfrage	Typisches Ergebnis	Artefakt in SDD
1	Grundidee & Anforderungsanalyse	Was soll entstehen — und warum?	geklärte, überprüfbare Anforderungen	spec.md
2	Architektur & Design	Wie bauen wir es? Welche Struktur, welche Bausteine?	begründete Technologie- und Struktur-entscheidungen	plan.md
3	Implementierung (bei ML: + Modell-Evaluation)	Funktioniert jeder Baustein für sich?	lauffähiger Code; bei ML zusätzlich Modell-Kennzahlen	tasks.md + Code
4	Qualitätsbewertung / Verifikation	Erfüllt das Ganze die Anforderungen aus Phase 1?	bestandene Akzeptanzkriterien / Tests	Tests gegen spec.md
5	Deployment	Wie kommt es zu den Nutzenden?	ausgelieferte, nutzbare Anwendung	—
6	Betrieb & Weiterentwicklung (Rework)	Was muss sich ändern, ohne Bestehendes zu brechen?	Wartung, neue Features, nächste Iteration	STATUS.md , neue Feature- spec.md

Die rechte Spalte zeigt, dass SDD keine Alternative zum SDLC ist, sondern **eine konkrete Art, ihn zu gehen**: Für die frühen Phasen legt es je eine eigene Datei an, statt alles zu vermischen.

3 | Drei Dinge, die man leicht falsch macht

Anforderung und Entwurf nicht vermischen (Phase 1 ≠ Phase 2). Das *Was* („der Nutzer soll X können“) ist unabhängig davon, *wie* man es baut. Dieselbe Anforderung lässt sich oft auf mehrere Arten umsetzen — verschiedene Techniken, verschiedene Architekturen. Wer beides in einen Topf wirft, legt sich zu früh technisch fest und merkt Denkfehler zu spät. Merksatz: *Eine Anforderung, mehrere mögliche Pläne.*

Es gibt zwei Ebenen der Prüfung (Phase 3 vs. Phase 4). In Phase 3 fragt man: *Funktioniert der einzelne Baustein?* (Bei ML-Projekten gehört hier die Modell-Evaluation hin: Ist das Modell treffsicher genug?) In Phase 4 fragt man etwas anderes: *Erfüllt die fertige Anwendung als Ganzes die Anforderungen aus Phase 1?* Beides kann auseinanderfallen — ein exzellenter Baustein in einer Anwendung, die am Ziel vorbeigeht, ist trotzdem ein Fehlschlag.

Der SDLC ist ein Kreis, keine Linie. Nach Phase 6 ist ein Projekt nicht „fertig“. Jede Wartung, jede Erweiterung, jeder Fehler stößt die frühen Phasen im Kleinen erneut an: neue Anforderung → neuer Entwurf → Umsetzung → Prüfung. Ein neues Feature ist damit kein Neubau, sondern eine **neue Runde** im selben Zyklus.

4 | SDD als ein Weg durch den SDLC

Den SDLC kann man auf verschiedene Weisen gehen. Klassisch **wasserfallartig** (streng eine Phase nach der anderen, später Testen), als **V-Modell** (jede Entwurfs-Phase hat eine spiegelbildliche Test-Phase) oder **iterativ/agil** (viele kleine Runden statt eines großen Durchlaufs).

Spec-Driven Development ist eine moderne, KI-gestützte Variante: iterativ wie agile Ansätze, mit fest getrennten Artefakten für die frühen Phasen, und — wie im V-Modell und bei Test-Driven Development — die Tests leiten sich direkt aus den Anforderungen ab und werden *vor* dem Code geschrieben. Der Gewinn: Der Zustand des Projekts liegt in Dateien, nicht nur im Kopf oder im Kontextfenster eines Modells.

5 | Transfer-Aufgabe

Jetzt übertragen Sie das Gerüst auf Ihr aktuelles Projekt.

(a) Füllen Sie die Tabelle für Ihr laufendes Projekt aus:

#	Phase	Was haben wir hier konkret getan?	Beleg / Artefakt	Noch offen?
1	Grundidee & Anforderungsanalyse			
2	Architektur & Design			
3	Implementierung (+ Modell-Eval.)			
4	Qualitätsbewertung			
5	Deployment			
6	Betrieb & Weiterentwicklung			

(b) Beantworten Sie in je ein bis zwei Sätzen:

1. In **welcher Phase** steht Ihr Projekt gerade?
2. Welche Phase war bei Ihnen bisher am **dünnsten** ausgeprägt — und warum?
3. Ihr **nächster geplanter Schritt**: In welcher Phase liegt er? Was löst er in den *früheren* Phasen wieder aus?

Frage 3 ist der eigentliche Kern: Achten Sie darauf, ob Ihr nächster Schritt eine *neue Runde* im Zyklus startet — und wo genau sie beginnt.